

Resolución de KenKen de punta a punta mediante Visión Computacional y Programación con Restricciones

Rody Sebastian Vilchez Marin (U202216562), José Javier Villanueva Aramburu (U202311894),
Jorge Rolando García Roca (U202211490)
Grupo 3 ù Tópicos en Ciencias de la Computación (IACC0058)
Universidad Peruana de Ciencias Aplicadas (UPC)
Lima, Perú

Resumen—Presentamos un sistema que lee un tablero de KenKen desde una imagen y lo resuelve. La primera fase localiza y rectifica el tablero, deduce su tamaño por la periodicidad de la rejilla, separa las jaulas midiendo el grosor de cada arista y lee las etiquetas por correlación con plantillas tipográficas, sin aprendizaje automático. La segunda formula el puzzle como un modelo de Programación con Restricciones parametrizado en n y lo resuelve con CP-SAT. Sobre 70 tableros reales publicados por KrazyDad, de 4×4 a 9×9 y con la verdad extraída del texto vectorial de los propios PDF, la lectura de etiquetas alcanza un 97.1 % (895 etiquetas, κ de Cohen de 0.969) y 48 tableros se leen correctamente de punta a punta. En los otros 22 el sistema no devuelve ninguna solución errónea: 3 lecturas se rechazan por una etiqueta ilegible y en 19 el solver demuestra que la instancia leída no tiene solución. El mismo mecanismo detectó un error en la verdad extraída de un PDF. Resolver un tablero de hasta 9×9 tarda menos de 50 ms.

Index Terms—KenKen, Programación con Restricciones, CP-SAT, Visión Computacional, OCR

I. INTRODUCCIÓN

KenKen es un cuadrado latino de $n \times n$ dividido en *jaulas*: grupos de celdas contiguas con una operación aritmética y un objetivo que los valores de la jaula deben cumplir. Resolverlo desde una imagen exige dos cosas de naturaleza distinta: percibir la estructura del tablero (tamaño, jaulas y etiquetas) y razonar sobre ella.

Separamos esas dos cosas con un contrato explícito: la fase de visión produce una *instancia* (tamaño y lista de jaulas con sus celdas, operación y objetivo) y la fase de razonamiento solo ve esa instancia. Las contribuciones son tres:

- un pipeline de visión sin aprendizaje que detecta las jaulas por el grosor de las aristas;
- un modelo de Programación con Restricciones (CP) parametrizado en n que, además de resolver, comprueba si la solución es única;
- una evaluación sobre 70 tableros reales con verdad exacta, que muestra que un conjunto sintético sobreestima la precisión y que el solver actúa como verificador de la lectura.

Trabajo relacionado. Mulamba *et al.* [1] estudian la resolución de restricciones basada en percepción sobre imágenes

de sudoku: una red neuronal interpreta la imagen y el solver razona sobre sus predicciones. Proponen inferencia conjunta, en la que el solver elige la asignación más probable entre las que cumplen las reglas, y analizan qué hacer cuando lo percibido viola las restricciones. Nuestro sistema toma un camino más simple: la visión entrega una sola lectura y el solver la acepta o la rechaza. Los resultados de la Sección V muestran que ese rechazo ya es informativo, y la inferencia conjunta es el paso natural siguiente (Sección VII).

II. DATOS

No encontramos un conjunto de datos público y etiquetado de tableros KenKen. El único precedente de KenKen por visión que hallamos (370 glifos reales) no declara licencia y trae los recortes estirados a 28×28 , lo que destruye la proporción del glifo; se descartó. Trabajamos con dos fuentes propias.

Tableros reales. Cinco cuadernillos de KrazyDad [2]: 4×4 fácil y difícil, 6×6 fácil y difícil, y 9×9 difícil. Un extractor recorta cada tablero de la página, lo rectifica a 1000×1000 píxeles y recorta la etiqueta de cada jaula: 72 tableros (48, 16 y 8 por tamaño) y 895 etiquetas. Como los PDF son vectoriales, cada etiqueta está como texto con su caja; proyectándola con la homografía del tablero se obtiene su celda, y con ella una verdad exacta sin etiquetar nada a mano. La celda sale de la geometría y no del detector de jaulas, así que el detector no se mide contra sí mismo. Dos tableros quedaron sin verdad por un fallo del extractor, de modo que la evaluación de punta a punta usa 70. Los primeros experimentos (Tabla II) se hicieron sobre un subconjunto inicial de 16 tableros de los cuadernillos de 6×6 y 9×9 difíciles, rasterizados a 150 ppp. KrazyDad autoriza el uso escolar y prohíbe el comercial; los PDF se descargan bajo demanda y no se redistribuyen.

Tableros sintéticos. Un generador parte de un cuadrado latino aleatorio, lo divide en jaulas de hasta cuatro celdas, les asigna una operación compatible y dibuja el tablero con distintas tipografías, opcionalmente con perspectiva, iluminación desigual y ruido. Da verdad gratis y cubre condiciones que los PDF no tienen, pero, como se ve en la Sección V, no basta para medir.

III. FASE 1: DE LA IMAGEN A LA INSTANCIA

III-A. Localización y rectificación

La imagen en escala de grises se binariza con umbral adaptativo, que tolera una iluminación desigual. Del contorno externo de mayor área se obtiene, por aproximación poligonal, un cuadrilátero cuyas esquinas definen una homografía que rectifica el tablero a un cuadrado [3]. Todo lo posterior trabaja sobre esa imagen rectificada, nunca sobre la original.

III-B. Tamaño del tablero

En un KenKen impreso la línea fina se interrumpe donde la atraviesa una jaula, y un sesgo de medio grado la parte en tramos a distintas alturas, así que no se puede exigir que cada línea cruce el tablero. Lo que sí se conserva es la periodicidad. Para cada candidato n se predicen las líneas interiores de ambos ejes y se puntúa la tinta de la *más débil*; con el n correcto esa línea conserva alrededor del 29 % de su longitud en tinta y con uno equivocado cae a cero. Se elige el mayor n con todas sus líneas presentes: los divisores del n correcto también pasan la prueba (una rejilla de 3 cae sobre la mitad de las líneas de un tablero de 6), pero solo la más fina las explica todas.

III-C. Jaulas

Conocida la rejilla se sabe dónde está cada arista interior, y basta medir su grosor. Se toma la mediana, a lo largo de la arista, del ancho del trazo de tinta *más cercano* a la posición prevista. La mediana descarta los cruces con otras líneas y con los dígitos, y buscar el trazo más cercano, en vez de exigir tinta en el píxel exacto, absorbe el error de la rectificación. Los grosores son bimodales porque el puzzle se imprime con dos anchos de pluma; el umbral entre fino y grueso sale del método de Otsu [4], con el marco exterior como referencia de grueso. Las celdas que no separa un borde grueso se agrupan con una estructura de unión-búsqueda.

III-D. Etiquetas

La etiqueta se imprime en la esquina superior izquierda de la celda *ancla* de la jaula, la primera en orden de fila y columna. El margen del recorte no es fijo: se mide dónde termina la línea de la rejilla y se deja un respiro proporcional. Con un margen fijo del 10 % de la celda se cortaba la parte superior de los dígitos (un 40 se leía 4U), y era el mayor error de todo el pipeline.

Los caracteres se separan por componentes conexas, reagrupando las que se solapan en horizontal porque el signo $\div$ se imprime como tres manchas. Cada carácter se clasifica por correlación normalizada contra plantillas renderizadas en siete tipografías, conservando la proporción del glifo. Probamos añadir rasgos estructurales, como el número de huecos cerrados, para separar 3 de 8, y la precisión sintética *bajó* de 99.4 % a 95.4 %; lo que sí resolvió esas confusiones fue ampliar el banco de tipografías.

III-E. Validación

Antes de llegar al solver la instancia se valida: las jaulas deben particionar el tablero y ser conexas, y las de resta y división deben tener exactamente dos celdas. Una lectura que viola el contrato se rechaza aquí y no llega a la Fase 2.

IV. FASE 2: MODELO DE PROGRAMACIÓN CON RESTRICCIONES

IV-A. Variables y dominios

Sea X una matriz de variables enteras de $n \times n$ con

$$x_{i,j} \in \{1, 2, \dots, n\} \quad \forall i, j \in \{1, \dots, n\}.$$

El modelo se construye a partir de la instancia y no contiene ningún dato de un tablero concreto: el mismo código resuelve cualquier n y cualquier partición en jaulas.

IV-B. Cuadrado latino

Cada fila y cada columna es una permutación de $1..n$, expresada con la restricción global `AllDifferent`:

$$\text{ALLDIFFERENT}(x_{i,1}, \dots, x_{i,n}),$$

$$\text{ALLDIFFERENT}(x_{1,j}, \dots, x_{n,j})$$

para todo i, j : $2n$ restricciones globales en total.

IV-C. Restricciones de jaula

Para cada jaula C_k con variables V_k , operación op_k y objetivo T_k :

- **Igualdad** ($=$): $v = T_k$, con $V_k = \{v\}$.
- **Suma** ($+$): $\sum_{v \in V_k} v = T_k$, una restricción lineal.
- **Producto** ($\times$): $\prod_{v \in V_k} v = T_k$, con la restricción global de multiplicación de CP-SAT.
- **Resta** ($-$): con $V_k = \{a, b\}$, $\text{máx}(a, b) - \text{mín}(a, b) = T_k$.
- **División** ($\div$): con $V_k = \{a, b\}$, $\text{máx}(a, b) = T_k \cdot \text{mín}(a, b)$.

En resta y división el orden en que la visión lista las dos celdas no significa nada, y el par $\text{máx} / \text{mín}$ hace el modelo simétrico. Cada una de esas jaulas añade dos variables auxiliares, $M_k = \text{máx}(a, b)$ y $m_k = \text{mín}(a, b)$, con dominio $1..n$. Para la división basta la igualdad del producto: con dominios enteros positivos implica que el cociente es exacto.

IV-D. Restricciones reificadas

La formulación directa de la resta es una disyunción, $(a - b = T_k) \vee (b - a = T_k)$, que se modela reificando cada lado en una variable booleana: $\beta_k \Rightarrow a - b = T_k$ y $\neg \beta_k \Rightarrow b - a = T_k$ (en CP-SAT, `OnlyEnforceIf`). Lo mismo vale para la división. No la usamos: el par $\text{máx} / \text{mín}$ expresa la misma relación con dos restricciones globales que el solver propaga directamente, sin variables booleanas ni ramificación sobre el orden de las celdas. Ninguna otra restricción del modelo es condicional, así que no hay otro lugar donde la reificación haga falta.

Donde sí será necesaria es en la reparación guiada por el solver (Sección VII): si la visión ofrece varias lecturas candidatas para una etiqueta, cada candidata c de la jaula k

lleva su literal $\ell_{k,c}$, se impone $\sum_c \ell_{k,c} = 1$ y la restricción de la jaula se activa con $\ell_{k,c} \Rightarrow \text{jaula}_k(c)$. Ese es el uso natural de la reificación en este problema: elegir qué lectura creer.

IV-E. Resolución y unicidad

El modelo se resuelve con CP-SAT [5], que combina propagación de restricciones con un núcleo SAT con aprendizaje de cláusulas. Además de buscar una solución, el sistema cuenta soluciones con un tope de dos, deteniendo la enumeración al llegar a él. Un puzzle publicado tiene solución única, así que una lectura que deja el tablero sin solución, o con más de una, delata un error de la Fase 1.

IV-F. Complejidad

El modelo tiene n^2 variables de dominio n , $2n$ restricciones AllDifferent, una restricción por jaula y dos variables auxiliares por cada jaula de resta o división. Su tamaño es $O(n^2)$, lineal en el número de celdas.

Resolverlo es otra cosa. El espacio de asignaciones tiene n^{n^2} puntos; las restricciones de cuadrado latino lo reducen al número de cuadrados latinos de orden n [6], que sigue creciendo de forma superexponencial: 576 para $n = 4$, unos 8.1×10^8 para $n = 6$ y unos 5.5×10^{27} para $n = 9$. Completar un cuadrado latino parcial ya es NP-completo [7], y KenKen comparte esa estructura, así que no cabe esperar una garantía de tiempo polinomial. En la práctica, las jaulas de un puzzle publicado restringen tanto que la propagación de AllDifferent y de las restricciones aritméticas poda casi todo el árbol: como muestra la Sección V-E, ningún tablero de hasta 9×9 tarda más de 50 ms. El conteo con tope dos cuesta, en el peor caso, una búsqueda completa adicional: para demostrar que no hay una segunda solución, o que no hay ninguna, el solver tiene que agotar el árbol.

V. EVALUACIÓN

V-A. Lectura de etiquetas sobre 70 tableros reales

Sobre las 895 etiquetas del conjunto real, la lectura completa (número y operación) coincide con la verdad en el 97.09 % de los casos. Como el reparto de etiquetas está muy desbalanceado, medimos también el acuerdo con el κ de Cohen [8], que descuenta el acuerdo esperable por azar: $\kappa = 0.969$, “casi perfecto” en la escala de Landis y Koch [9].

La Tabla I agrupa los 26 fallos. Cuatro de cada cinco son la misma confusión, un 5 leído como 6 (21 de 26), y 17 de ellos caen en la etiqueta 5+, muy frecuente en los tableros de 4×4 . Los tres casos ilegibles son etiquetas de cuatro cifras que desbordan su celda y dejan el operador fuera del recorte. El error, por tanto, no está repartido: se concentra en un par de glifos y en un caso geométrico, y ambos tienen arreglo dirigido.

La misma prueba, ejecutada en otra máquina con otro sistema operativo, dio 95.98 % y $\kappa = 0.957$. El banco de plantillas se renderiza con las tipografías instaladas en el sistema, que es la causa más probable de la diferencia; las cifras de este informe son las de la máquina descrita en la Sección V-E.

Tabla I
FALLOS DE LECTURA SOBRE 895 ETIQUETAS REALES

Verdad	Leído	Casos
5+	6+	17
15+, 15*, 35*	16+, 16*, 36*	4
2400*, 4800*, 2160*	ilegible	3
6*, 288*	8*, 286*	2
Total		26

Tabla II
EXACTITUD DE CUATRO OCR SOBRE 412 ETIQUETAS REALES (%)

Método	Etiqueta	Operación	Número
Plantillas (este trabajo)	98.8	99.3	98.8
Tesseract 5.3.4	97.1	97.8	97.8
RapidOCR (PP-OCR)	64.8	68.4	94.2
TrOCR base printed	60.9	63.6	95.1

V-B. Elección del OCR

Sobre el subconjunto inicial de 16 tableros comparamos las plantillas con tres motores genéricos, con las mismas 412 etiquetas (Tabla II): Tesseract [10], RapidOCR, que ejecuta PP-OCR [11] sobre ONNX Runtime, y TrOCR [12]. Plantillas y Tesseract no se distinguen: discrepan en 11 etiquetas, 9 a favor de las plantillas y 2 de Tesseract, con $p = 0.065$ en la prueba exacta de McNemar [13]. Los dos superan con claridad a RapidOCR y TrOCR ($p < 0.001$), que leen bien los números pero no los operadores.

Más que el orden, importa que los dos mejores fallan en etiquetas distintas: al menos un motor acierta 411 de las 412 etiquetas (99.8 %). Ese es el techo de un jurado de motores.

V-C. Lo que el conjunto sintético escondía

En el conjunto sintético (80 tableros, n de 4 a 7; Tabla III) la Fase 1 llegó al 100 % en tableros limpios con tipografías del banco. Al estrenar los tableros reales falló en los 16 primeros: la rejilla real se interrumpe, la tinta no cae en el píxel predicho, el margen fijo cortaba los dígitos y KrazyDad imprime glifos que el banco no tenía. Los cuatro problemas son supuestos que el generador nunca puso a prueba. La tabla muestra además el peso de la tipografía: con una tipografía que el banco no conoce, la instancia completa cae al 81.0 % en limpio y al 52.4 % en foto. El conjunto sintético sirve para desarrollar, pero las cifras que valen son las de los tableros reales.

V-D. De punta a punta

Pasamos cada uno de los 70 tableros con verdad por el sistema completo, cotejamos la instancia leída con la del PDF y resolvimos lo leído (Tabla IV). El tamaño n es correcto en los 67 tableros que producen una instancia, y la partición en jaulas coincide con la verdad en 66; la excepción es el tablero cuya verdad resultó errónea, que se discute abajo. Con otra rasterización, en el subconjunto inicial de 150 ppp, el detector fundía dos jaulas en 6 de 16 tableros; en las imágenes de 1000×1000 no ocurre. Los fallos que quedan son de etiqueta, los de la Tabla I.

Tabla III
FASE 1 SOBRE 80 TABLEROS SINTÉTICOS (%)

Condición	Tipografía	Etiquetas	Instancia completa
Limpio	En el banco	100.0	100.0
Foto	En el banco	99.8	96.6
Limpio	No vista	99.4	81.0
Foto	No vista	95.9	52.4

Tabla IV
DE PUNTA A PUNTA SOBRE 70 TABLEROS REALES

n	Tableros	Bien leídos	Rechazados	Sin solución
4	47	36	0	11
6	16	9	1	6
9	7	3	2	2
Total	70	48	3	19

Lo importante es cómo falla: en ninguno de los 22 tableros mal leídos el sistema devuelve una solución errónea. En 3 una etiqueta resulta ilegible y la instancia se rechaza antes del solver; en los otros 19 la instancia es válida pero el solver demuestra que no tiene solución. En estos tableros, un solo dígito mal leído bastó para que el cuadrado latino dejara de cerrar.

El mismo mecanismo encontró un error en la verdad. En un tablero de 9×9 la instancia extraída del PDF no tiene solución, mientras que la lectura del sistema tiene una única; la diferencia es una jaula de división que el extractor no registró. Como un puzzle publicado tiene solución, el error está en la verdad y no en la lectura, y por eso ese tablero cuenta como bien leído en la Tabla IV. Con 70 tableros no es una garantía, pero muestra que el modelo CP sirve para verificar tanto la lectura como los datos con que se la evalúa.

V-E. Rendimiento del solver

La Tabla V da el tiempo de pared de `resolver`, incluida la construcción del modelo, sobre 30 instancias sintéticas por tamaño. Las instancias sintéticas no garantizan unicidad: miden el solver, no la dificultad de un puzzle publicado. Sobre la verdad de los 70 tableros reales, la mediana es de 4.4 ms en 4×4 , 4.1 ms en 6×6 y 10.8 ms en 9×9 , con un máximo de 14.1 ms. Sobre lo que el sistema lee, incluidas las instancias sin solución, donde el solver tiene que agotar la búsqueda para demostrarlo, el máximo es de 20.4 ms. Medido en un procesador x86-64 de 14 hilos (Intel Core Ultra 7 255U) con OR-Tools 9.15 y Python 3.13.

VI. INTEGRACIÓN

Un CLI (`solve.py`) encadena las dos fases sin intervención manual: lee la imagen, construye la instancia, la valida, la resuelve y dibuja el tablero con la solución superpuesta. Si el solver no encuentra solución, lo informa y dibuja el tablero sin rellenar, en vez de inventar una.

VII. LIMITACIONES Y TRABAJO FUTURO

- **La confusión entre 5 y 6.** Explica 21 de los 26 fallos de etiqueta. Es un problema de un par de glifos, no del

Tabla V
TIEMPO DE RESOLVER EN INSTANCIAS SINTÉTICAS (MS)

n	Mediana	P95	Máximo
4	6.7	13.9	17.5
5	5.4	12.7	14.2
6	5.8	13.0	13.7
7	8.5	15.8	18.4
8	10.6	19.7	23.6
9	12.3	23.0	24.0

método, y se puede atacar con plantillas específicas de la tipografía de KrazyDad o con un clasificador dedicado a ese par.

- **Etiquetas que desbordan su celda.** Hay que detectar que falta el operador y ampliar el recorte solo en ese caso; ampliarlo siempre mete la línea vecina, que se lee como un 1.
- **Un solo editor y sin fotos.** Los 70 tableros reales son de un mismo editor y una misma tipografía, y son PDF rasterizados: tinta real pero sin sombras, curvatura de papel ni desenfoque. Falta un conjunto amplio de tableros impresos y fotografiados.
- **Jurado de motores y reparación guiada por el solver.** Hoy el solver rechaza una lectura errónea pero no la corrige. Donde los motores de OCR discrepan, o donde el clasificador duda entre 5 y 6, se pueden pasar las candidatas al modelo CP con restricciones reificadas y quedarse con la combinación que deja el tablero con solución única. Es la inferencia conjunta de Mulamba *et al.* [1], y los 19 tableros sin solución de la Tabla IV son exactamente los casos que aprovecharía.

VIII. DECLARACIÓN DE USO DE IA

El andamiaje de la Fase 1 (visión) se desarrolló con asistencia de IA generativa (Claude Code, de Anthropic). El modelo de Programación con Restricciones de la Fase 2 lo escribió el equipo, sin asistencia de IA. La revisión y la compilación de este informe, el script que mide el solver (`scripts/medir_solver.py`) y el de la evaluación de punta a punta (`scripts/punta_a_punta_scraped.py`) se hicieron con asistencia de Claude Code; un integrante usó Gemini, de Google, para analizar la salida del benchmark de OCR. Todas las cifras salen de scripts del repositorio y se pueden volver a obtener, y el equipo las revisó.

REFERENCIAS

- [1] M. Mulamba, J. Mandi, A. . Mahmutoullar y T. Guns, “Perception-based constraint solving for sudoku images,” *Constraints*, 2024, doi: 10.1007/s10601-024-09372-9.
- [2] KrazyDad, “Inkies” (cuadernillos 4×4 , 6×6 y 9×9 , Volume 1, Book 1), KrazyDad.com. [En línea]. Disponible: <https://krazydad.com/inkies/>. Consultado: 18 de septiembre de 2026.
- [3] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [4] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.
- [5] L. Perron y F. Didier, “CP-SAT,” Google OR-Tools, v9.15, 2026. [En línea]. Disponible: https://developers.google.com/optimization/cp/cp_solver/

- [6] OEIS Foundation, "A002860: Number of Latin squares of order n ," *The On-Line Encyclopedia of Integer Sequences*. [En línea]. Disponible: <https://oeis.org/A002860>
- [7] C. J. Colbourn, "The complexity of completing partial Latin squares," *Discrete Applied Mathematics*, vol. 8, no. 1, pp. 25–30, 1984.
- [8] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [9] J. R. Landis y G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
- [10] R. Smith, "An overview of the Tesseract OCR engine," en *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [11] Y. Du *et al.*, "PP-OCR: A practical ultra lightweight OCR system," arXiv:2009.09941, 2020.
- [12] M. Li *et al.*, "TrOCR: Transformer-based optical character recognition with pre-trained models," en *Proc. AAAI Conf. Artificial Intelligence*, 2023.
- [13] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.